

智能感知与自动驾驶技术

谭国平

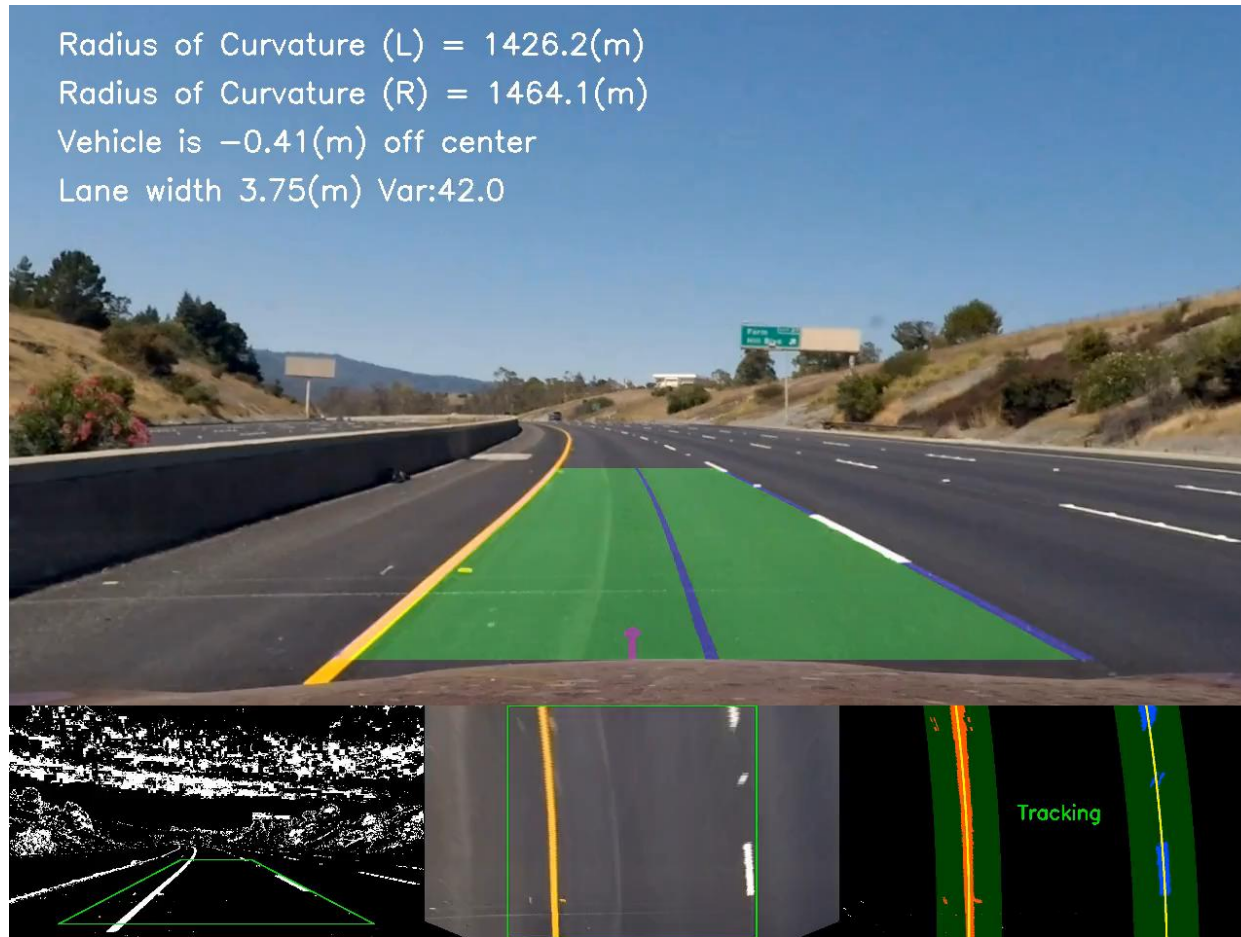
gptan@hhu.edu.cn

河海大学计算机与软件学院

智能感知与自动驾驶技术

- 第1章 绪论
 - 第2章 视觉传感器
 - **第3章 自动驾驶车道线检测**
 - 第4章 自动驾驶目标检测
 - 第5章 自动驾驶决策控制
-

任务目标：基于计算机视觉的车道线检测



第3章 基于计算机视觉的车道线检测

- 3.1 OpenCV图像处理基础
- 3.2 车道线检测 – 图像处理方法
- 3.2 车道线检测 – 深度学习方法

3.1 OpenCV图像处理基础

本节主要介绍基于OpenCV的图像处理基础方法和函数。

讨论主题：

图像读取与显示

颜色空间转换

仿射变换

透视变换

阈值处理

图像滤波

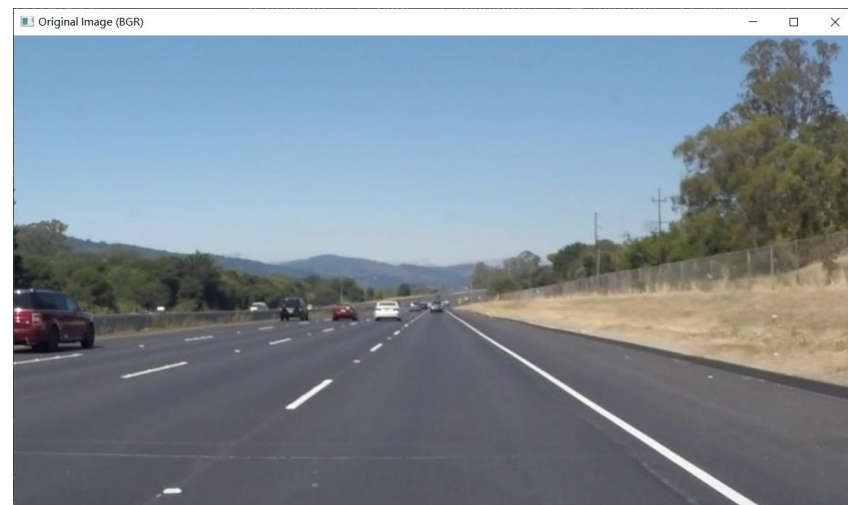
...

OpenCV 读取显示图像

```
import cv2
import numpy as np

# 读取图像
# OpenCV 默认以 BGR 格式读取图像
img_path = r'D:\AIR\Teaching\solidWhiteRight.jpg'
img_bgr_original = cv2.imread(img_path)

# 直接使用 OpenCV 显示 BGR 图像
cv2.imshow('Original Image (BGR)', img_bgr_original)
cv2.waitKey(0) # 等待按键
cv2.destroyAllWindows() # 关闭所有 OpenCV 窗口
```



1. 颜色空间转换

“颜色空间”提供不同的表示方法，适用于不同的任务。

1. BGR/RGB: 由蓝(Blue)、绿(Green)、红(Red)三个通道组成。(OpenCV默认BGR)
2. Grayscale(灰度图): 只有亮度信息, 没有颜色信息。常用于边缘检测、特征提取等。
3. HSV(Hue,Saturation,Value): Hue-色调, Saturation-饱和度, Value-明度。常用于图像分割和识别特定颜色的物体。

颜色空间转换原理:

颜色空间转化是通过特定的数学公式将像素值从一个空间的表示映射到另一个空间的表示。例如, RGB图像→灰度图; RGB→HSV; HSV→RGB等。

RGB图像→灰度图常用算法:

浮点算法: $\text{Gray} = R \cdot 0.3 + G \cdot 0.59 + B \cdot 0.11$

整数方法: $\text{Gray} = (R \cdot 30 + G \cdot 59 + B \cdot 11) / 100$

移位方法: $\text{Gray} = (R \cdot 76 + G \cdot 151 + B \cdot 28) \gg 8$

平均值法: $\text{Gray} = (R + G + B) / 3$

仅取绿色: $\text{Gray} = G$

1. 颜色空间转换

```
img_bgr = img_bgr_original.copy() # 使用副本  
# 1. BGR 转换为灰度图  
img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)  
# 2. BGR 转换为 HSV  
img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
```



2. 几何变换 – 仿射变换

作用：

仿射变换是一种保持直线和平行性的几何变换，它包括旋转、平移、缩放和剪切等操作。仿射变换操作由原图中三个点及其在目标图像中对应的三个点确定。

原理：

使用 `cv2.getAffineTransform()` 从三对点计算2x3的变换矩阵 `M`，同样使用 `cv2.warpAffine()` 应用变换。

算法：

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & c & b_x \\ b & d & b_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

常见仿射变换类型：

1.平移 (Translation)：图像在二维平面上沿着x轴和y轴方向移动一定距离。

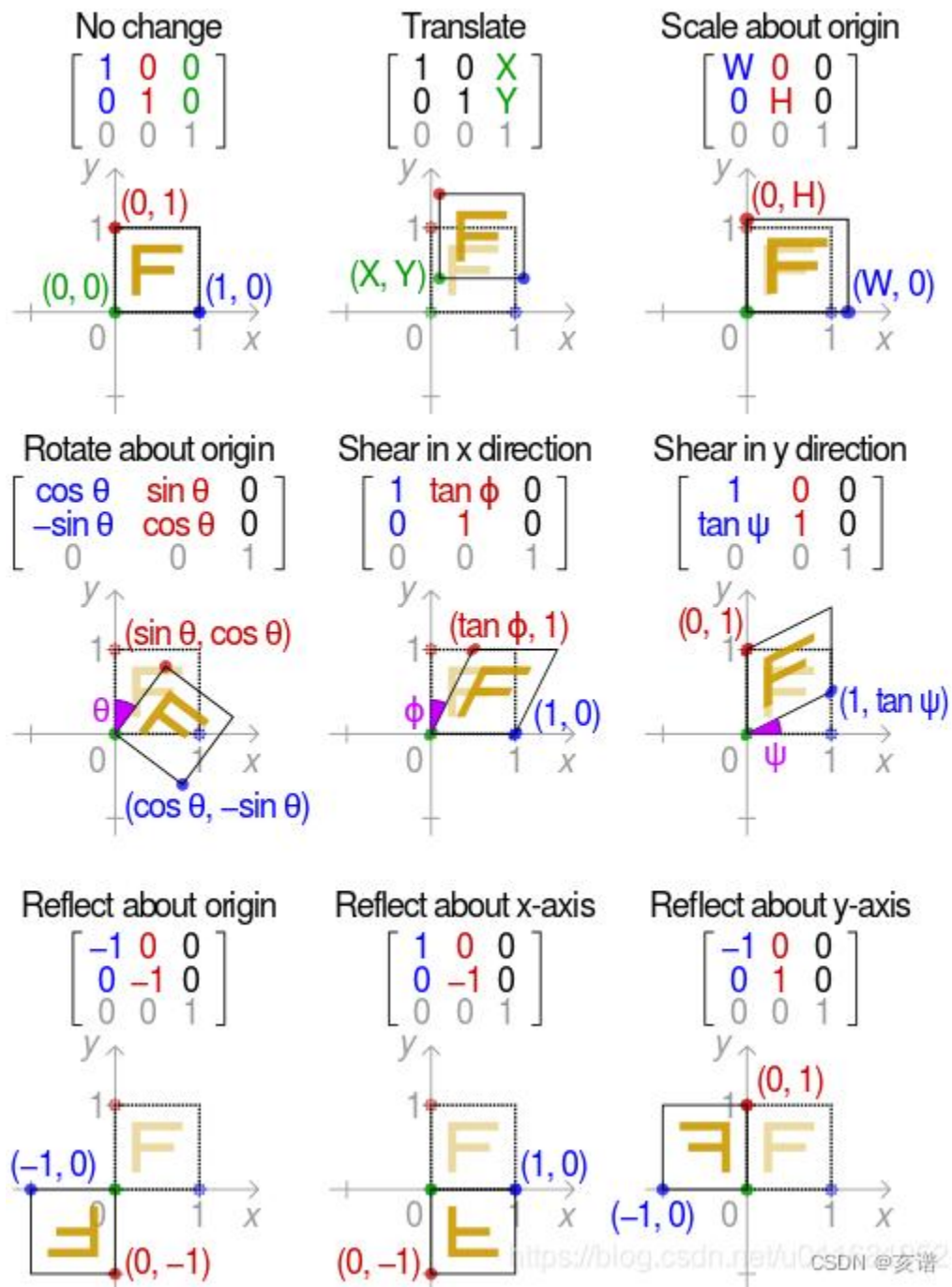
2.缩放 (Scaling)：图像的大小按照一定的比例因子放大或缩小，可以是各向同性的（在所有方向上按相同比例缩放）或各向异性的（在不同方向上按不同比例缩放）。

3.旋转 (Rotation)：图像围绕一个固定点（通常是图像中心）旋转一定角度。

4.剪切 (Shearing)：图像沿着某一轴方向进行倾斜变换，导致直线变为非垂直或非水平的直线。

5.反射 (Reflection)：图像关于某一轴（如x轴或y轴）进行镜像反射。

6.投影变换 (Projection)：图像被投影到一个虚拟的视角上，可以模拟三维效果。



2.1 几何变换 – 平移

作用：

将图像在X和Y方向上移动。

原理：

平移需要一个2x3的变换矩阵 $M = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}$ 。 t_x 是水平位移， t_y 是垂直位移。使用 `cv2.warpAffine()` 应用变换。

算法：

2.1 几何变换 – 平移

```
img_bgr = img_bgr_original.copy()
rows, cols = img_bgr.shape[:2]
tx = 50 # 水平平移50像素
ty = 100 # 垂直平移100像素
M_translate = np.float32([[1, 0, tx], [0, 1, ty]])
img_translated = cv2.warpAffine(img_bgr, M_translate, (cols, rows))
```



2.2 几何变换 - 缩放

作用：

改变图像的尺寸大小，通过“插值”算法计算新像素的值。

原理：

缩放时，如果新尺寸与原尺寸不成整数倍，就需要“插值”来决定新像素的值。常见的插值方法有 `cv2.INTER_NEAREST`, `cv2.INTER_LINEAR` (默认), `cv2.INTER_CUBIC`。

插值算法：

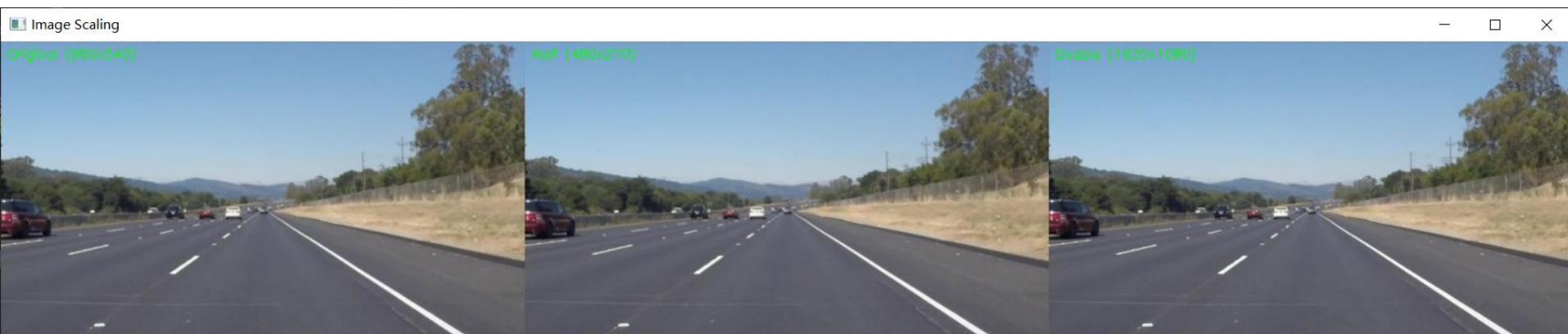
- 1. `cv2.INTER_NEAREST`（最邻近插值）：**直接取最接近的像素值作为新像素的值。计算速度快，但可能会在图像中产生明显的锯齿效应，尤其是在图像缩小时。
- 2. `cv2.INTER_LINEAR`（双线性插值）：**考虑新像素位置周围 4 个最邻近的像素值，然后通过线性加权平均的方式来计算新像素的值。这种方法比最邻近插值平滑得多，但依然是一种相对简单的插值方法。
- 3. `cv2.INTER_CUBIC`（双三次插值）：**考虑新像素位置周围 16 个像素点的值，然后通过三次多项式函数来计算新像素的值。这种方法在图像放大时特别有效，可以产生更加平滑和自然的边缘。

2.2 几何变换 - 缩放

```
img_bgr = img_bgr_original.copy()

# 缩小到一半 (指定目标尺寸 dsize)
img_scaled_half = cv2.resize(img_bgr, (width // 2, height // 2),
interpolation=cv2.INTER_LINEAR)

# 放大两倍 (使用缩放因子 fx, fy)
img_scaled_double = cv2.resize(img_bgr, None, fx=2, fy=2,
interpolation=cv2.INTER_CUBIC)
```



2.3 几何变换 – 旋转

作用：

围绕指定中心点旋转图像。

原理：

使用`cv2.getRotationMatrix2D()`构建旋转矩阵，需要入参：旋转中心、角度和缩放因子。同样使用`cv2.warpAffine()`应用变换。

平移算法：

2.3 几何变换 – 旋转

```
img_bgr = img_bgr_original.copy()
rows, cols = img_bgr.shape[:2]
center = (cols // 2, rows // 2) # 图像中心点
angle = 30 # 旋转角度
scale = 1.0 # 缩放因子
# 计算旋转矩阵
M_rotate = cv2.getRotationMatrix2D(center, angle, scale)
img_rotated = cv2.warpAffine(img_bgr, M_rotate, (cols, rows))
```



2.4 几何变换 – 通用仿射变换

```
img_bgr = img_bgr_original.copy()
rows, cols = img_bgr.shape[:2] # 保持原图像大小
# 原图中的三个点
pts1_affine = np.float32([[50, 50], [cols-50, 50], [50, rows-50]])
# 目标图像中的三个点
pts2_affine = np.float32([[50, 200], [cols-100, 80], [80, rows-20]])
# 计算仿射变换矩阵
M_affine = cv2.getAffineTransform(pts1_affine, pts2_affine)
# 应用仿射变换
img_affine_transformed = cv2.warpAffine(img_bgr, M_affine, (cols, rows))
```



2.5 几何变换 – 透视变换

作用：

可将图像的一个四边形区域映射到另一个四边形，不保持平行线关系。

方法：

使用 `cv2.getPerspectiveTransform()` 从四对点计算 3×3 的变换矩阵 M ，使用 `cv2.warpPerspective()` 应用变换。

算法：

2.5 几何变换 – 透视变换

```
img_bgr = img_bgr_original.copy()
rows, cols = img_bgr.shape[:2]

# 这些点需要你根据你的图像手动选取或通过其他方法检测
# 这里使用图像的角点和一些偏移作为示例
pts1_perspective = np.float32([
    [cols*0.1, rows*0.1], [cols*0.9, rows*0.2],
    [cols*0.2, rows*0.9], [cols*0.8, rows*0.8] ])

output_width = 300
output_height = 400
pts2_perspective = np.float32([
    [0, 0], [output_width - 1, 0],
    [0, output_height - 1], [output_width - 1, output_height - 1] ])

M_perspective = cv2.getPerspectiveTransform(pts1_perspective,
pts2_perspective)
img_perspective_transformed = cv2.warpPerspective(img_bgr, M_perspective,
(output_width, output_height))
```

2.5 几何变换 – 透视变换



3. 阈值处理

作用：

将灰度图像转换为二值图像（只有黑白两色）。

方法：

`cv2.threshold(src, thresh, maxval, type)`: 全局阈值。

`cv2.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType, blockSize, C)`: 自适应阈值，对光照不均的图像效果更好。

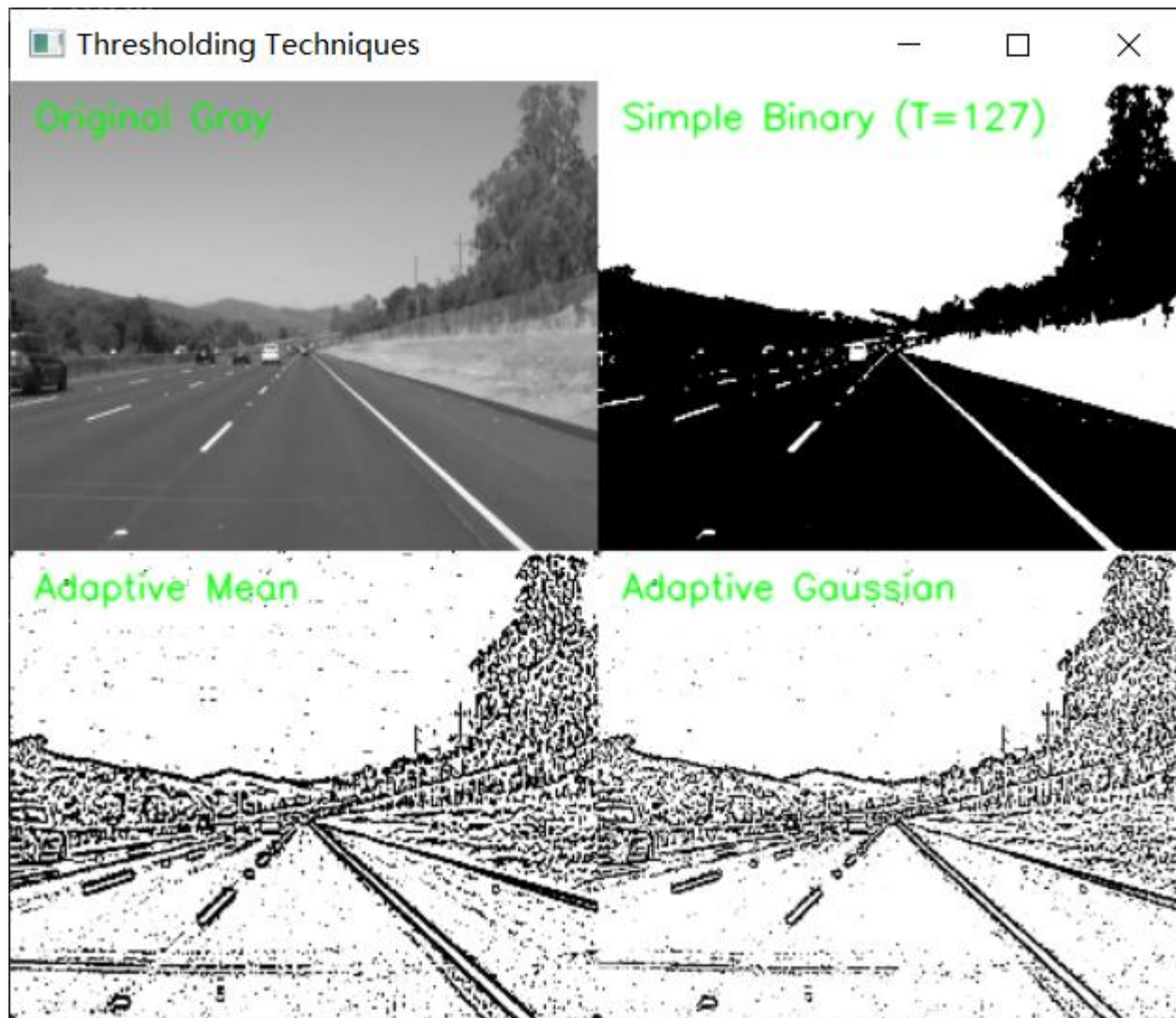
算法：

阈值处理

```
img_bgr = img_bgr_original.copy()
img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
# 应用不同的阈值处理方法
# 简单二值化
ret, thresh_binary = cv2.threshold(img_gray, 127, 255, cv2.THRESH_BINARY)

# 自适应阈值处理
# 对光照不均的图像效果更好
thresh_adaptive_mean = cv2.adaptiveThreshold(img_gray, 255,
cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY, 11, 2)
thresh_adaptive_gaussian = cv2.adaptiveThreshold(img_gray, 255,
cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)
```

阈值处理



4. 图像滤波

作用:

主要用于减少图像中的噪声。

方法:

大多基于“卷积”操作，用一个“核”滑过图像。

均值滤波 (cv2.blur()): 邻域像素平均值。

高斯滤波 (cv2.GaussianBlur()): 高斯核加权平均，更好地保留边缘。

中值滤波 (cv2.medianBlur()): 邻域像素中值，对椒盐噪声有效。

双边滤波 (cv2.bilateralFilter()): 考虑空间距离和像素值差异，去噪同时保边。

算法:

图像滤波

```
img_bgr = img_bgr_original.copy()

# 给原始 BGR 图像添加高斯噪声
noise = np.zeros(img_bgr.shape, np.uint8)
cv2.randn(noise, (0,0,0), (30,30,30))
img_noisy_bgr = cv2.add(img_bgr, noise)

# 滤波操作
img_mean_blur = cv2.blur(img_noisy_bgr, (5, 5))
img_gaussian_blur = cv2.GaussianBlur(img_noisy_bgr, (5, 5), 0)
img_median_blur = cv2.medianBlur(img_noisy_bgr, 5)
img_bilateral_blur = cv2.bilateralFilter(img_noisy_bgr, 9, 75, 75)
```

图像滤波



图像滤波



3.2 应用案例：车道线检测

本节主要介绍自动驾驶系统中经典车道线检测算法的处理流程。

讨论主题：

灰度化

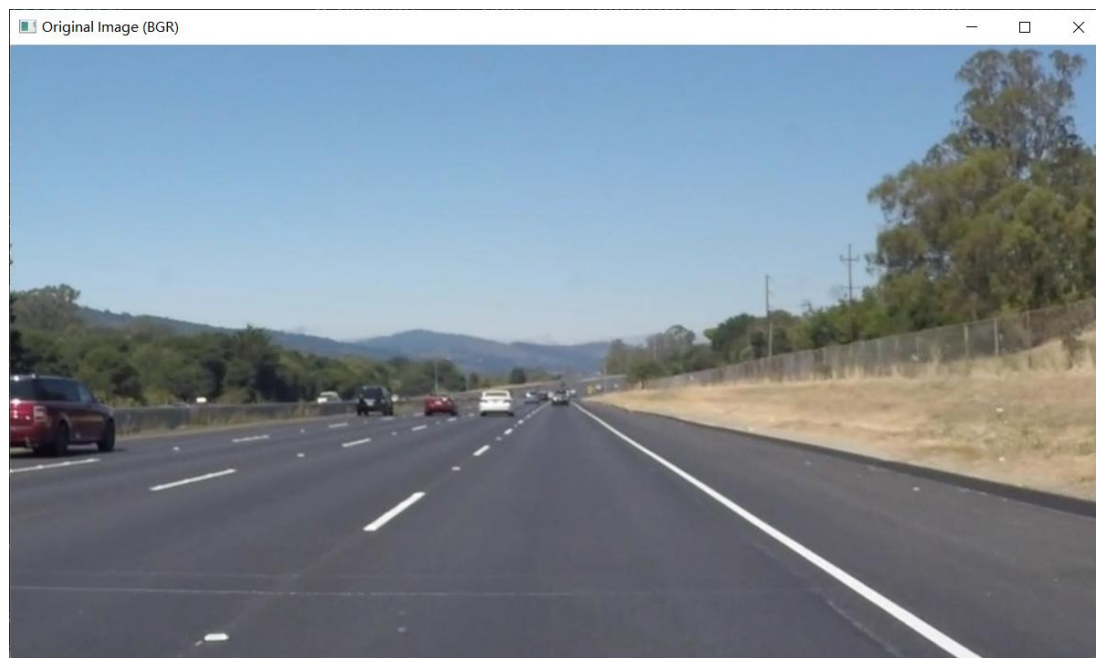
高斯模糊

Canny边缘检测

定义RoI

霍夫变换检测直线

...



车道线检测

0. 图像读取

读取原始图像并复制。

1. 图像灰度化

将图像从BGR（或RGB）颜色空间转换为灰度图像，因为边缘检测通常在灰度图像上进行。

2. 高斯模糊

- 使用高斯模糊对灰度图像进行平滑处理，减少噪声，为边缘检测做准备。



车道线检测

3. Canny边缘检测

使用 `cv2.Canny()` 函数检测图像中的边缘。利用参数 `low_threshold` 和 `high_threshold` 控制边缘连接的阈值。

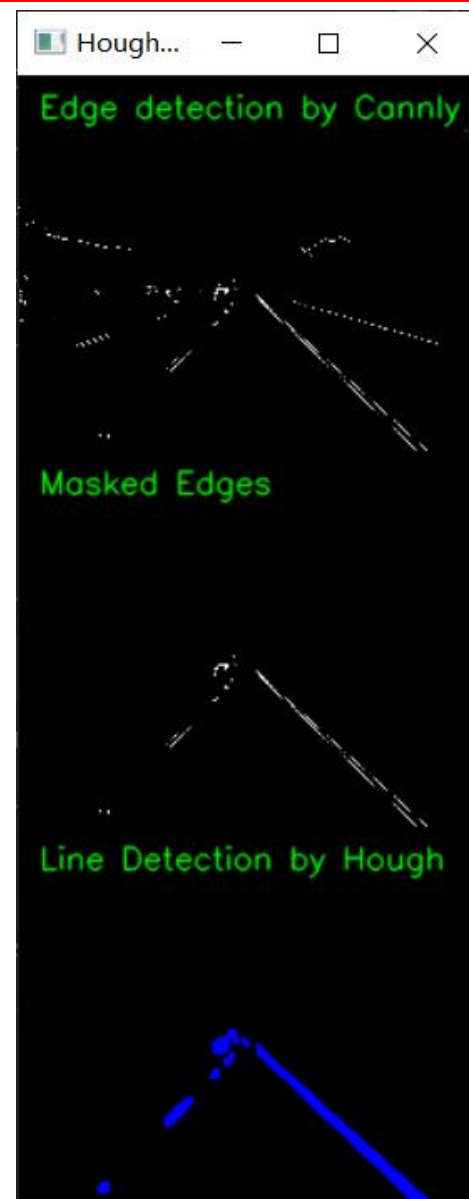
4. 创建掩膜

- 创建一个全黑的掩膜图像，与边缘图像大小相同。
- 定义一个多边形顶点数组，用于指定ROI。
- 使用 `cv2.fillPoly` 函数填充多边形区域。
- 使用 `cv2.bitwise_and` 应用掩膜，只保留多边形区域内的边缘。

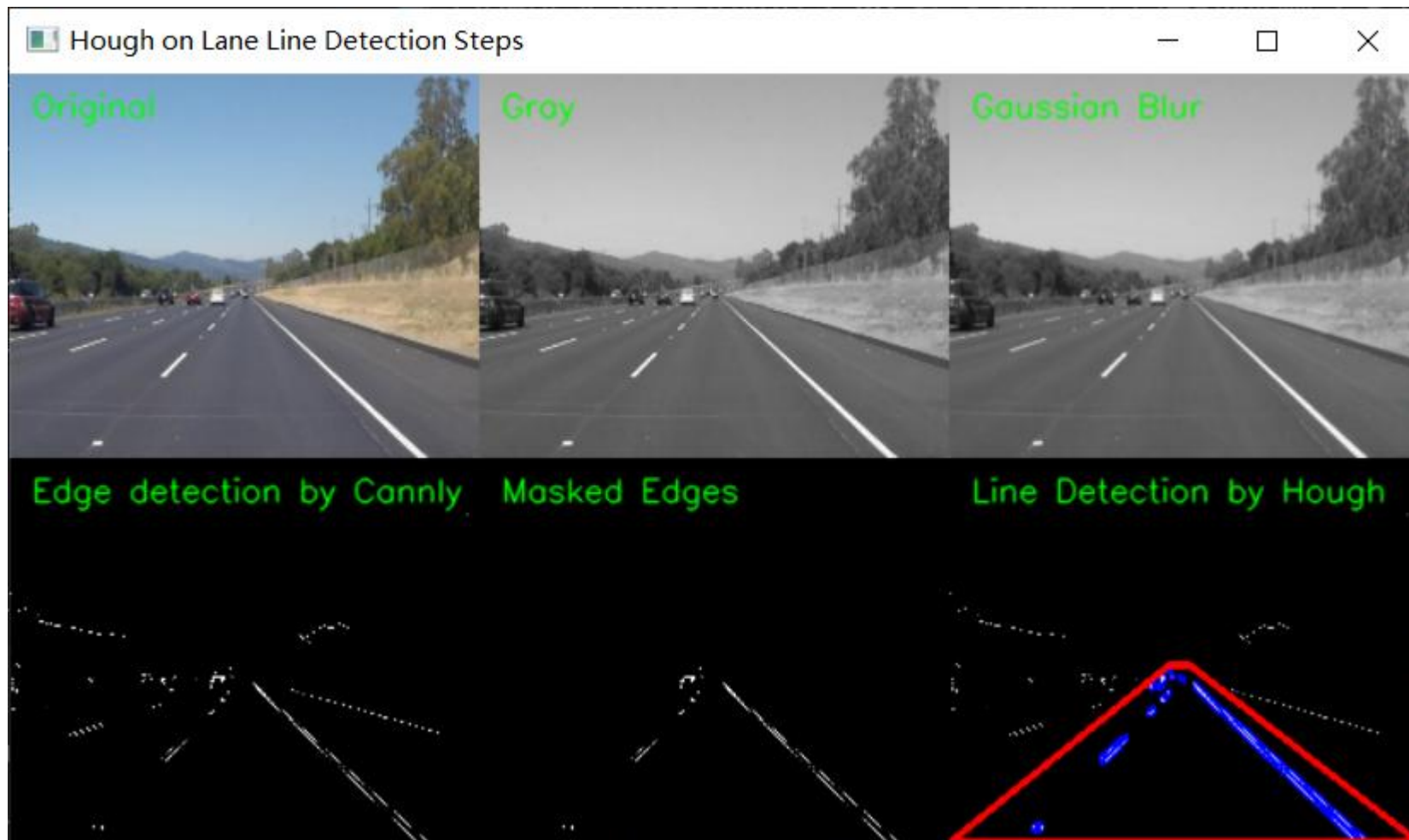
5. 霍夫变换检测直线

定义霍夫变换参数，包括分辨率、角度分辨率、阈值、最小线长度和最大线间隙。

使用 `cv2.HoughLinesP` 函数在边缘图像上检测直线。



车道线检测-主要处理流程



车道线检测-检测结果可视化

Input Image



Output Image [Lane Line Detected]



车道线检测-直道实时检测结果

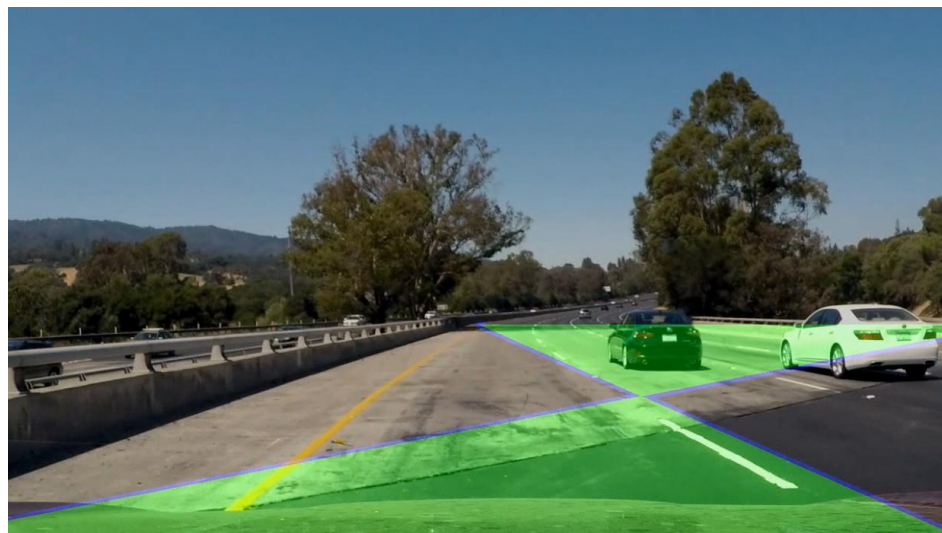


车道线检测-弯道实时检测问题



车道线检测-遗留问题

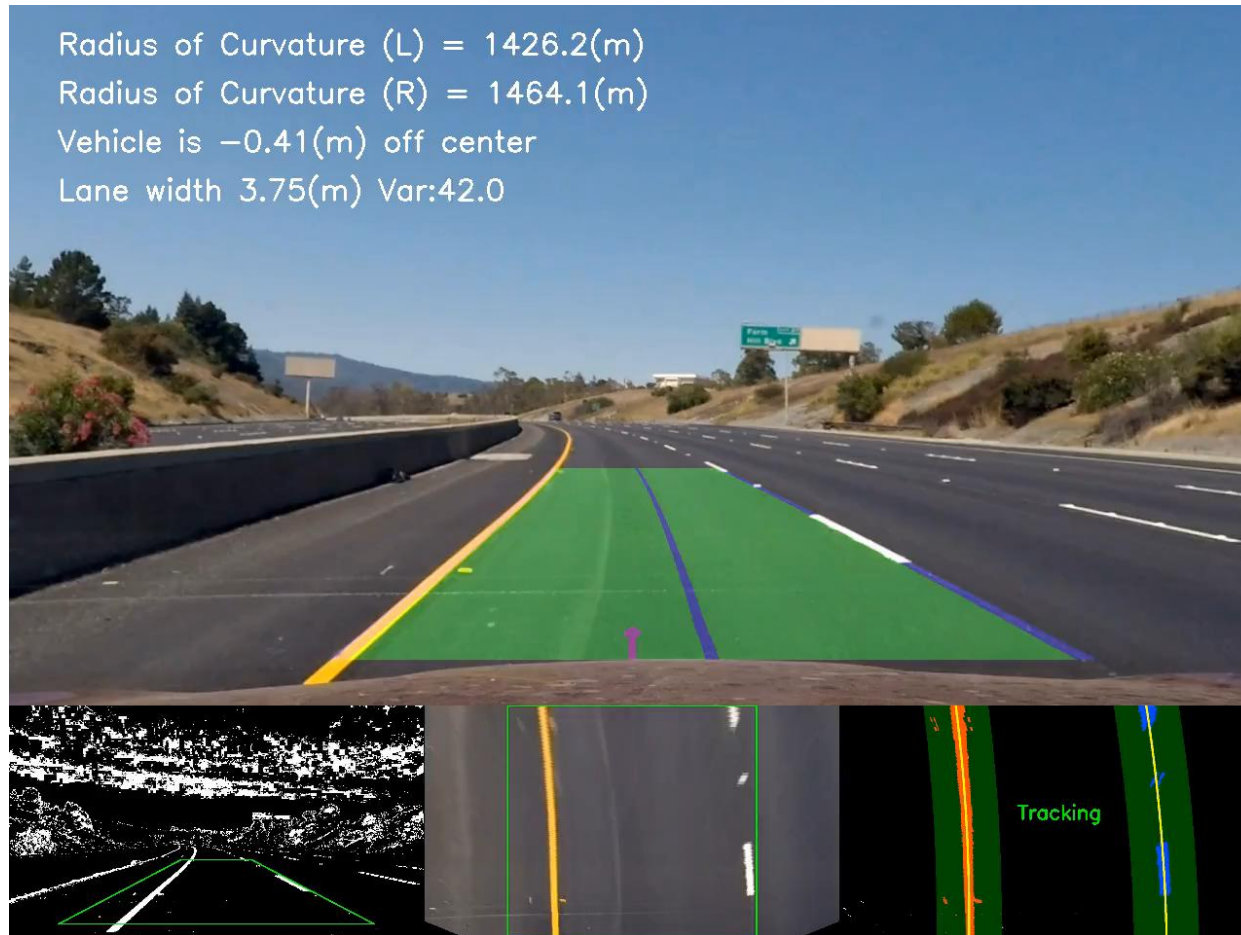
如何准确
检测弯道车道线？



非结构化道路如
何检测车道？

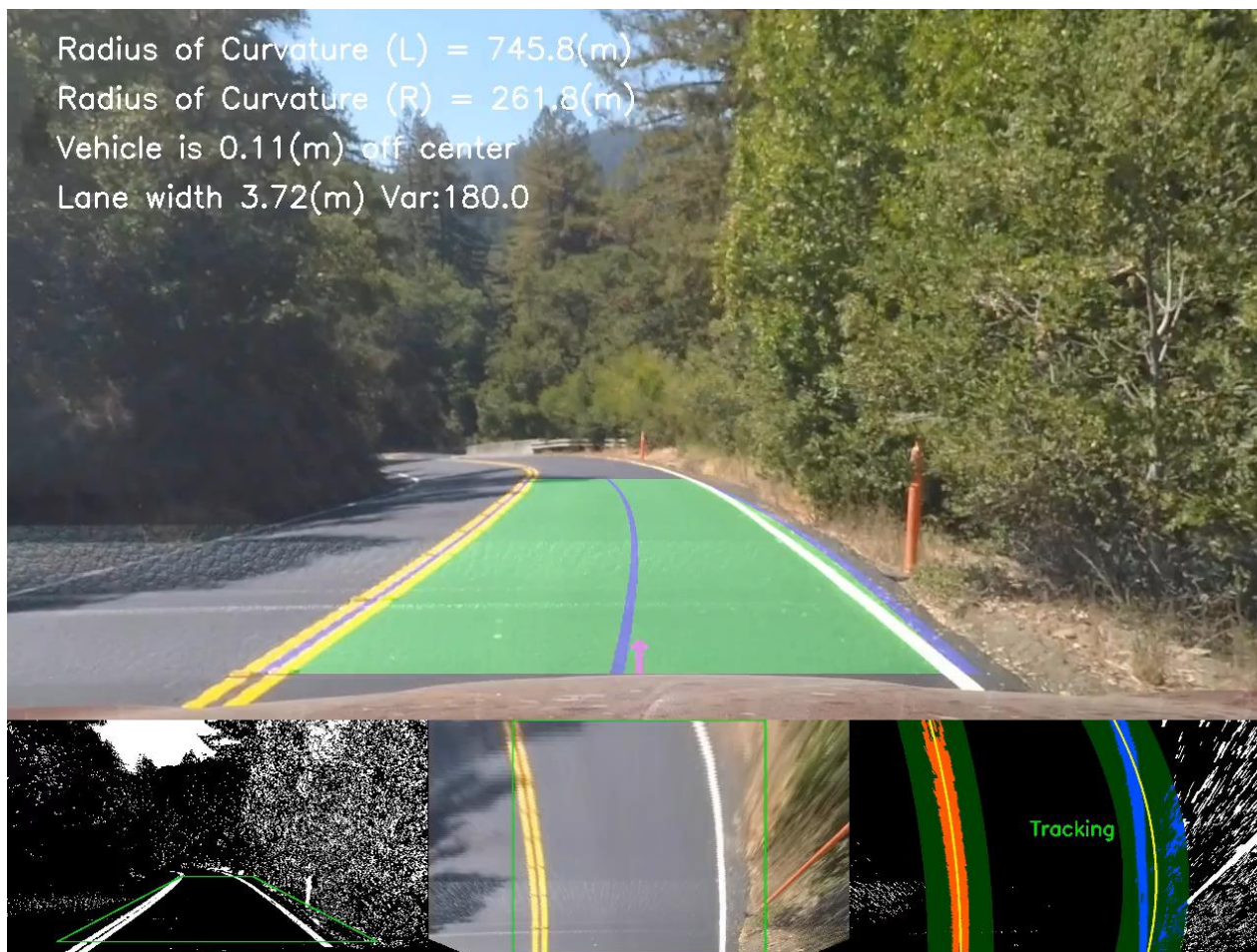


高级车道线检测



<https://www.kaggle.com/code/guopingtang/advanced-lane-detection>

高级车道线检测-遗留问题



<https://www.kaggle.com/code/guopingtang/advanced-lane-detection>

3.3 车道线检测 – 深度学习方法

本节主要介绍利用深度学习进行车道线检测的研究进展。

讨论主题：

数据集介绍

UNET语义分割

Ultra Fast Lane Detection

...

车道线检测数据集-TuSimple

拍摄介绍：主要采集区域国外高速公路上（安装在卡车的靠近车牌的位置），图像未畸变。

数据介绍：位于高速路，天气晴朗，车道线清晰；车道线以点来标注；车道线实际上不只是道路上的标线，虚线被当作了一种实线做处理的。双实线、白线、黄线这类信息没有标注。

图片大小：1280x720，图片数量：6408（训练集：3626；验证集：2782）。

标签格式：json，点形式标注。



test_set.clips.0530.1492626126171
818168_0.20.jpg.gt



test_set.clips.0530.1492626127172
745520_0.20.jpg.gt



test_set.clips.0530.1492626153155
598528_0.20.jpg.gt



test_set.clips.0530.1492626155156
451704_0.20.jpg.gt



test_set.clips.0530.1492626166147
797438_0.20.jpg.gt



test_set.clips.0530.1492626171146
236124_0.20.jpg.gt



test_set.clips.0530.1492626224112
349377_0.20.jpg.gt



test_set.clips.0530.1492626236105
069364_0.20.jpg.gt



test_set.clips.0530.1492626252095
353084_0.20.jpg.gt



test_set.clips.0530.1492626257094
533319_0.20.jpg.gt



test_set.clips.0530.1492626265087
865031_0.20.jpg.gt



test_set.clips.0530.1492626268085
982449_0.20.jpg.gt



test_set.clips.0530.1492626283076
624822_0.20.jpg.gt



test_set.clips.0530.1492626286076
989589_0.20.jpg.gt



test_set.clips.0530.1492626296068
790749_0.20.jpg.gt



test_set.clips.0530.1492626329051
620125_0.20.jpg.gt



test_set.clips.0530.1492626325035
311824_0.20.jpg.gt



test_set.clips.0530.1492626353036
397044_0.20.jpg.gt



test_set.clips.0530.1492626356032
639684_0.20.jpg.gt



test_set.clips.0530.1492626363029
955539_0.20.jpg.gt

车道线检测数据集-CULane

拍摄介绍：由不同驾驶员操控的六辆不同的轿车所安装的摄像头采集的，拍摄地点为北京城区道路和高速公路。该数据集采集了超过55小时的视频数据。

数据介绍：包含拥挤，黑夜，无线，暗影等八种难以检测的情况，最多标记4条车道线；

图片大小：1640x590；图片数量：133235（训练集：88880，验证集：9675，测试集：34680）。

标签：车道标注的就是当前车道和左右两车道（最多），不区分虚实线。

标签格式：txt点坐标格式



00000.jpg.gt



00030.jpg.gt



00060.jpg.gt



00090.jpg.gt



00150.jpg.gt



00180.jpg.gt



00210.jpg.gt



00240.jpg.gt



00300.jpg.gt



00330.jpg.gt



00360.jpg.gt



00390.jpg.gt



00450.jpg.gt



00480.jpg.gt



00510.jpg.gt



00540.jpg.gt

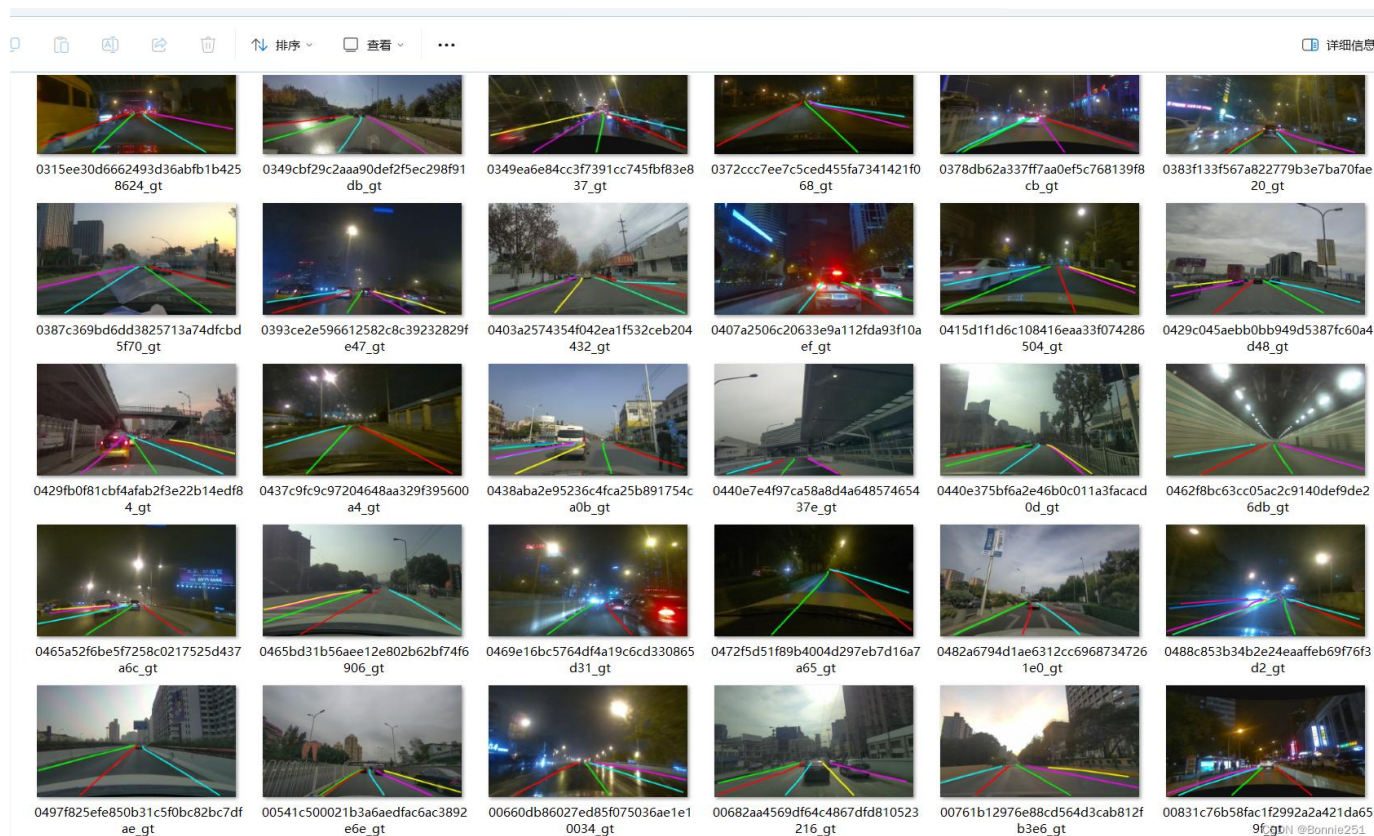
CSDN @Bonnie251

车道线检测数据集 - CurveLanes

拍摄介绍: 华为弯道检测数据集, 采用三次样条曲线手动标注所有车道线, 包括很多较为复杂的场景, 如S路、Y车道, 还有夜间和多车道的场景。

图片大小: 2650x144, **数量:** 分为训练集10万张, 验证集2万张, 测试集3万张;

标签格式: json格式。



车道线检测数据集 - LLAMAS

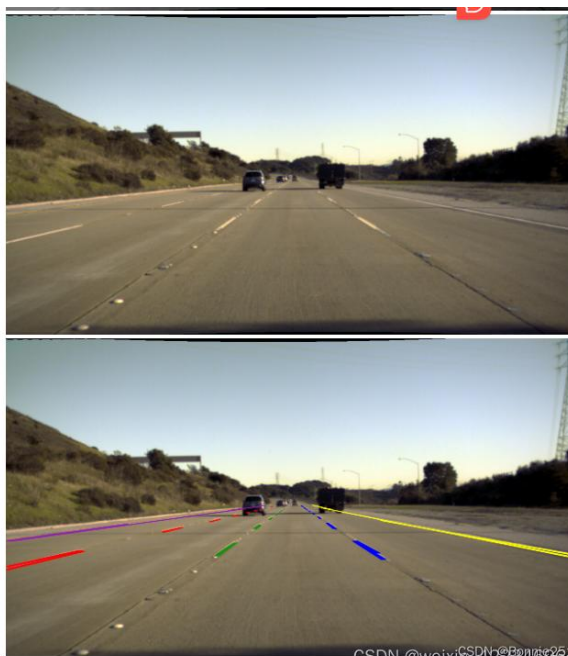
拍摄介绍：使用 14 条高速公路记录，每条大约 25 公里，收集传感器数据以在数据集中标记图像。国外高速。

标签：博世公司车道标记数据集,标注不是人工标注，而是高精地图生成的，全是高速场景。超过100米的注释，摄像头大致位于汽车外部的车牌处。提供原图的灰度图，总共4类。

图片大小：1276 x 717，图片数量：79113 (训练集: 58269 验证集: 20844 测试集: 20929(无标签))

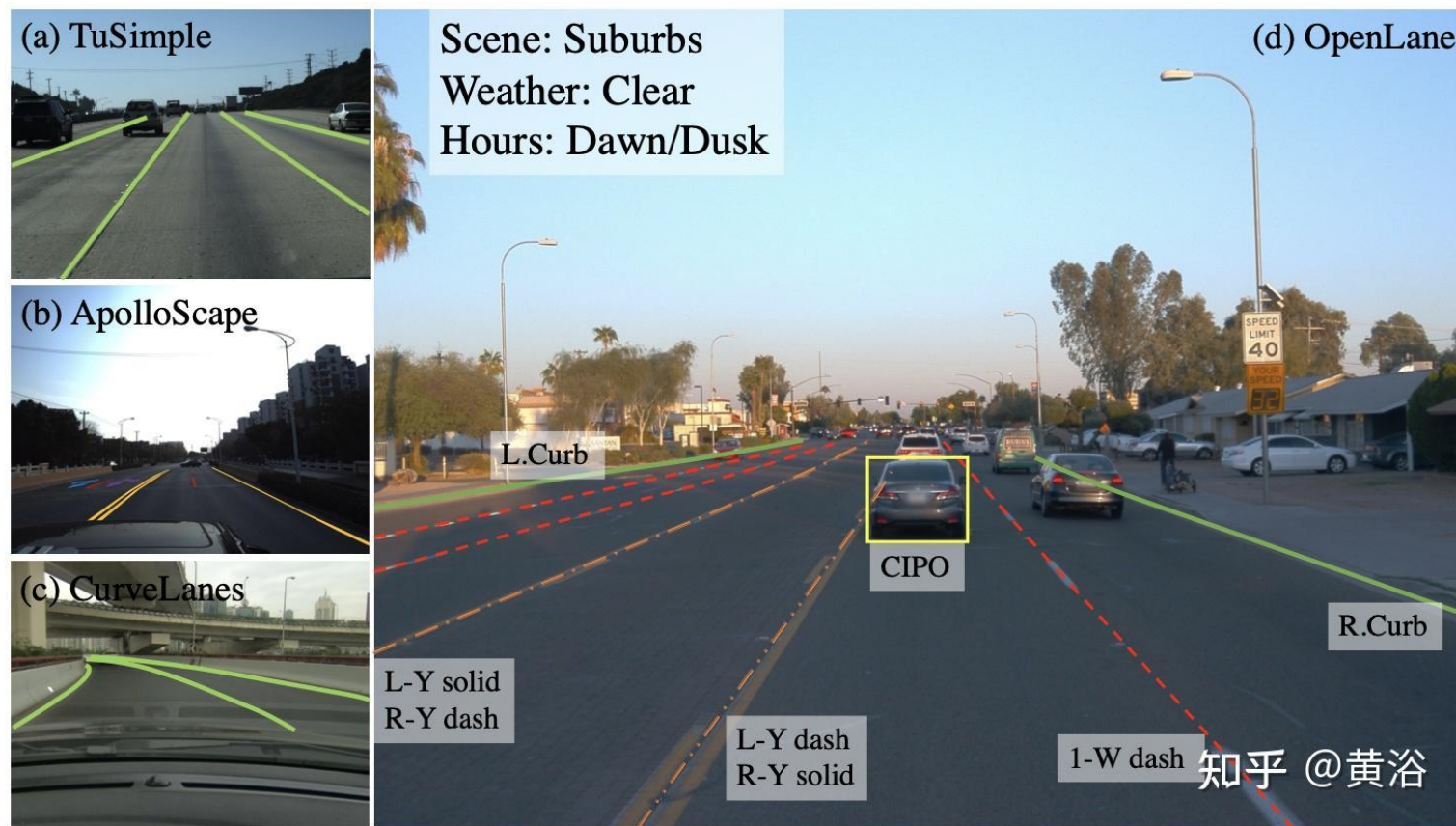
标签：左虚车道线，右虚车道线，左实车道线，右实车道线。

标签格式：以点json格式（三维坐标）。

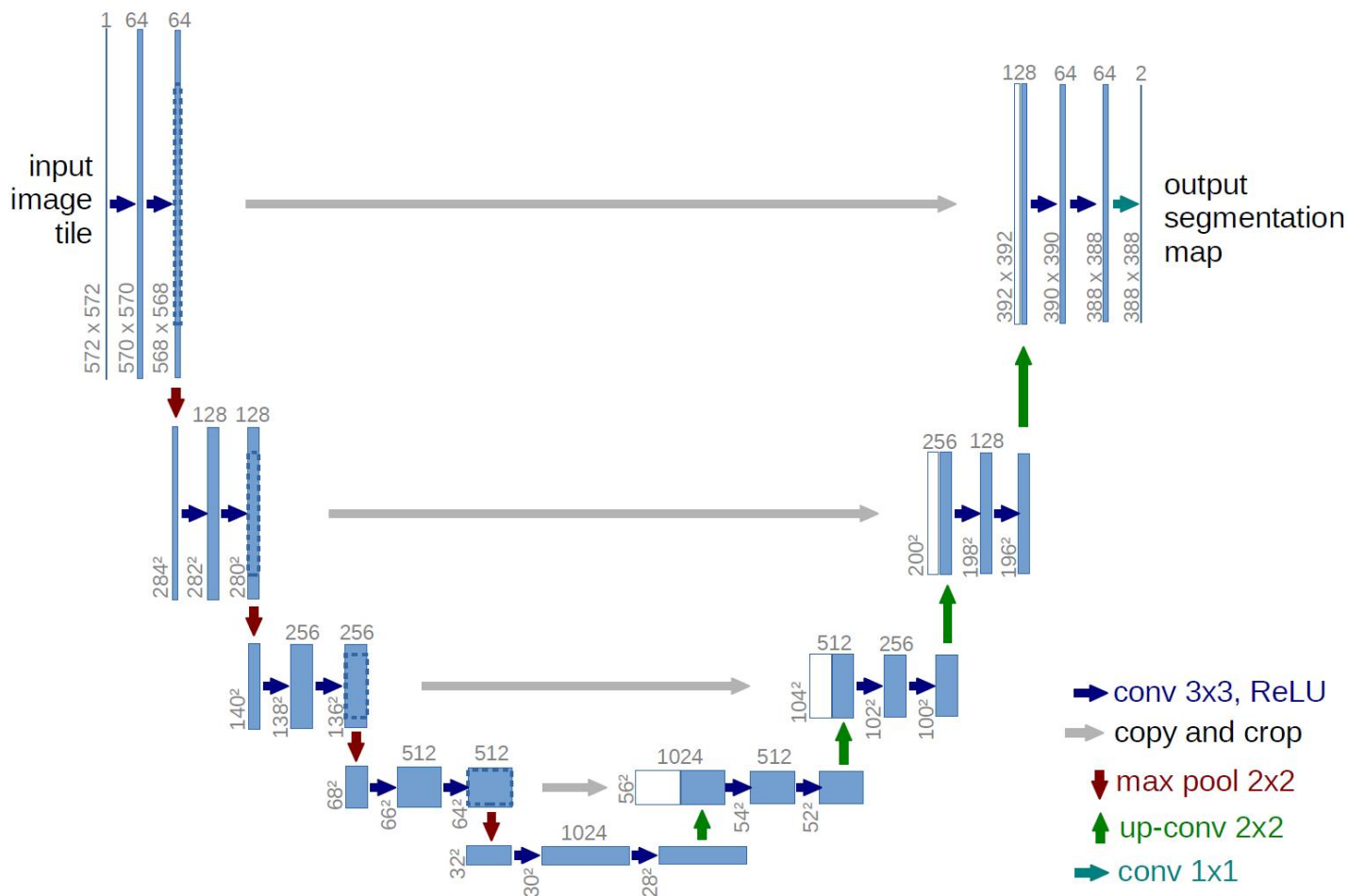


车道线检测数据集 - OpenLANE

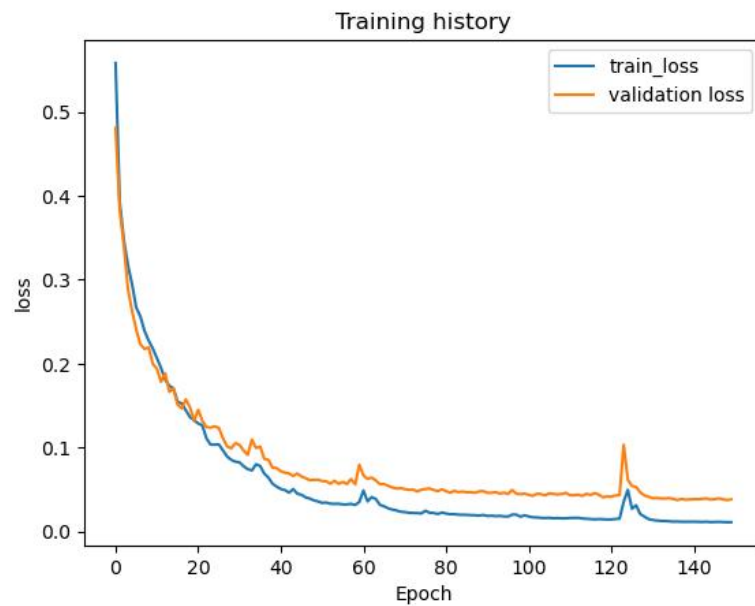
OpenLane 是迄今为止第一个真实世界和规模最大的 3D 车道数据集。数据集从公共感知数据集 Waymo Open Dataset 中收集有价值的内容，并为 1000 个路段提供车道和最近路径对象（CIPO）注释。简而言之，OpenLane 拥有 200K 帧和超过 880K 仔细注释的车道。



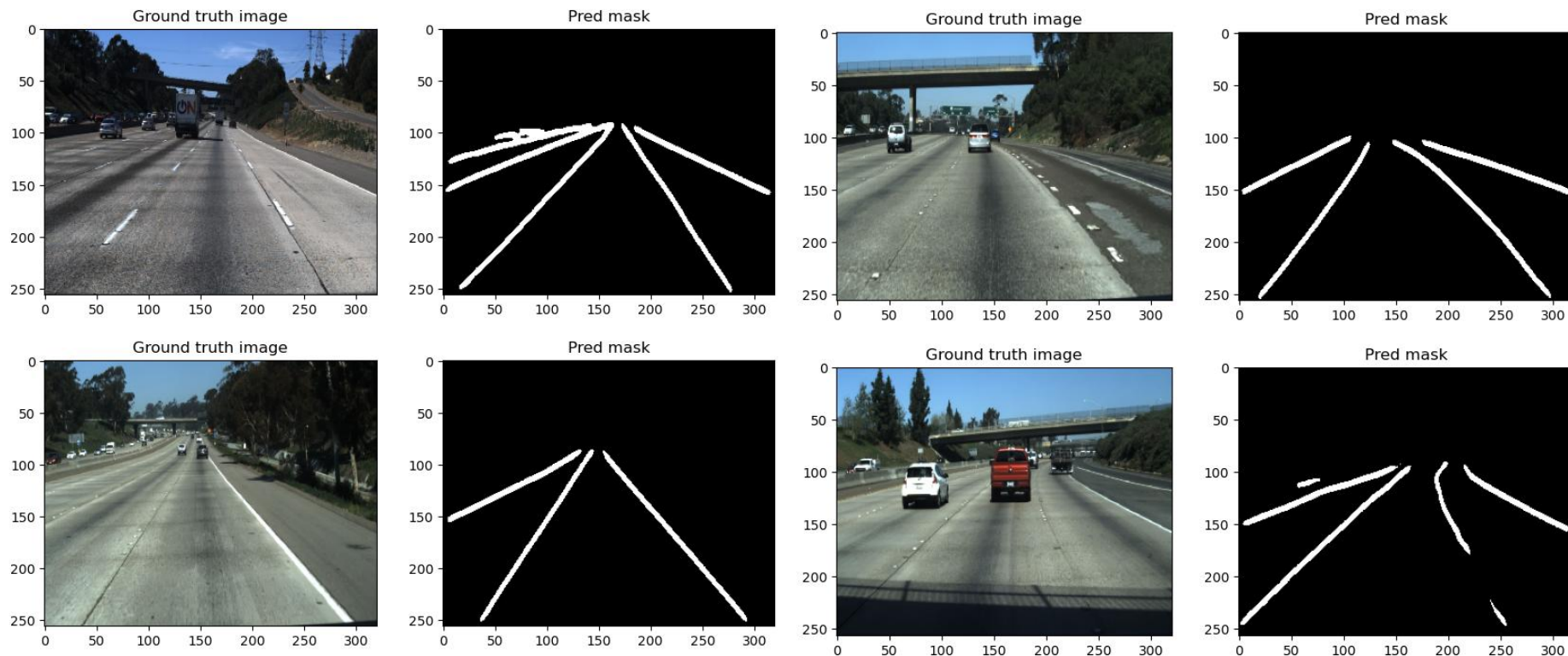
UNET语义分割-车道线检测



UNET语义分割-车道线检测

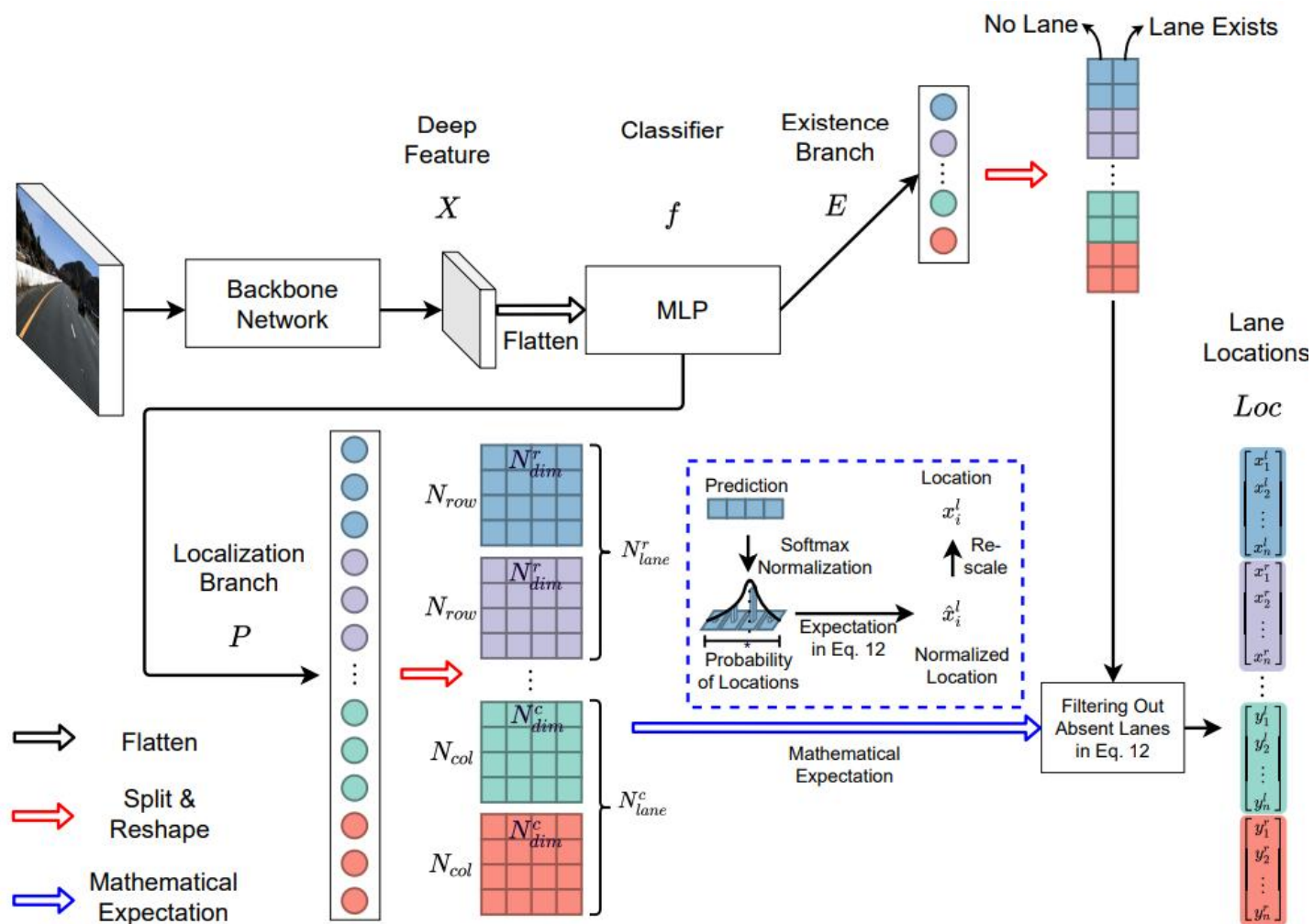


UNET语义分割-车道线检测



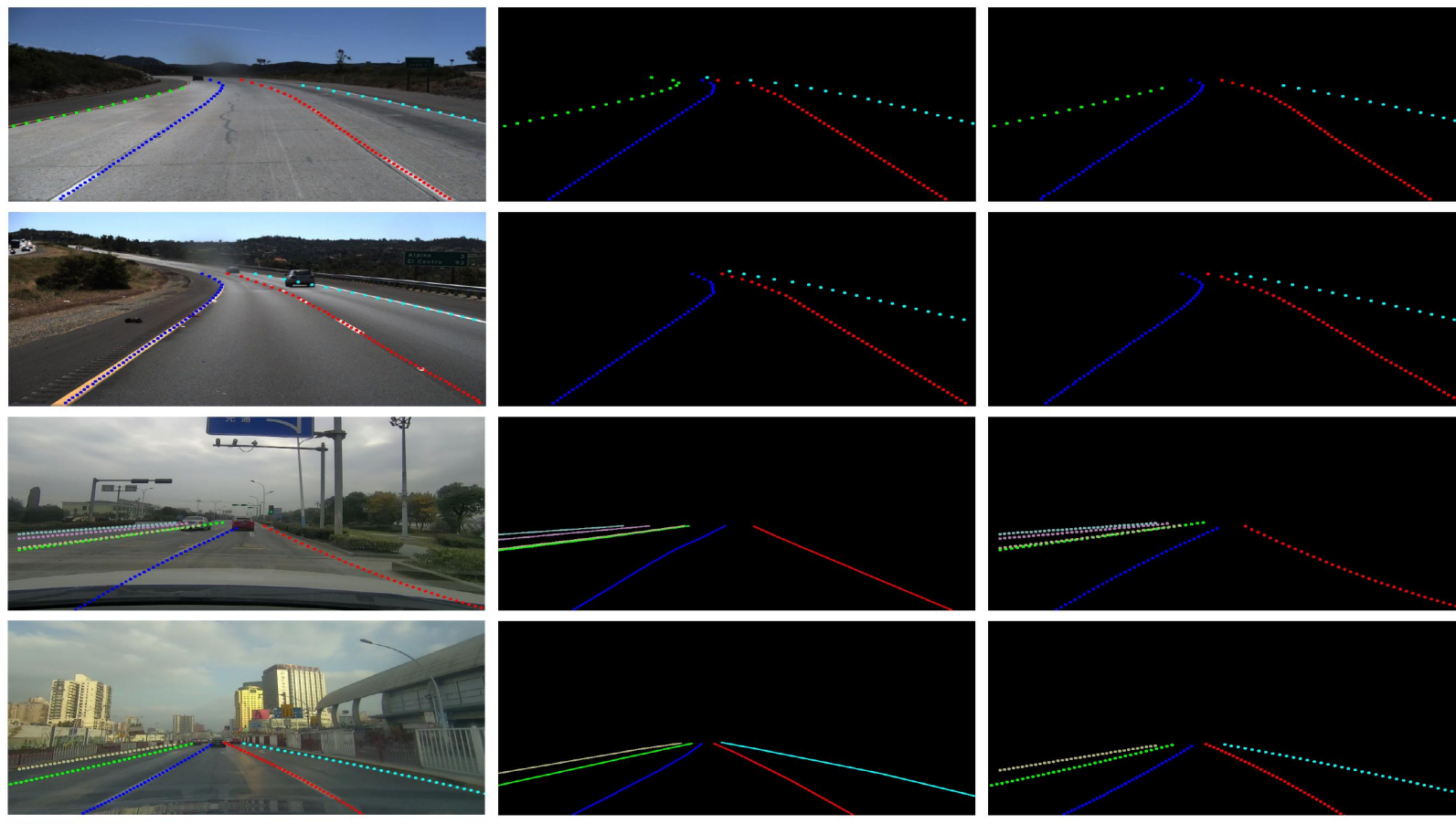
<https://www.kaggle.com/code/hikmatullahmohammadi/road-lane-detection-with-unet-tusimple-dataset>

Ultra-Fast-Lane-Detection



<https://github.com/cfzd/Ultra-Fast-Lane-Detection-v2>

Ultra-Fast-Lane-Detection



Visualization on the **CULane, TuSimple, CurveLanes, and LLAMAS datasets**. Each dataset is visualized with two rows. From left to right, the results are images, ground truths, and predictions, respectively. In the image, predicted lane points are annotated.

Ultra-Fast-Lane-Detection

